SADRŽAJ

1. UVOD 8
2. PREGLED PODRUČJA 9
2.1 Povijest 9
2.2 Usporedba 12
2.2.1 Microsoftovi aplikacijski okviri 12
2.2.2 Nezavisne biblioteke i aplikacijski okviri 12
3. FUNKCIONALNOST MFC-A 14
3.1 Windows programski model 14
3.2 Mapiranje poruka (Message Maps) 15
3.3 Hijerarhija klasa MFC-a 16
3.4 Dokument/pogled arhitektura 18
3.5 AFX funkcije 18
4. PODRŠKA U VISUAL STUDIJU 20
4.1 Integracija MFC-a u Visual Studio 20
4.2 MFC aplikacijski čarobnjak 20
4.3 Uređivači resursa 21
5. PRAKTIČAN RAD – UNO KARTAŠKA IGRA 22
5.1 Svrha aplikacije 22
5.2 Struktura aplikacije 22
5.3 Korištene MFC komponente 24
5.3.1 Dijaloški prozori 24
5.3.2 Afx funkcije 27
5.3.3 Dvostruko međuspremanje (Double Buffering) 28
5.3.4 Ostale komponente korištene u 'View' klasi 30
5.4 Korištenje GDI+ biblioteke 34
6. ZAKLJUČAK 35

LITERATURA 36
7. SAŽETAK 38
8. SUMMARY 39

POPIS SLIKA

Slika 1 MFC logo 9
Slika 2 Microsoft Office ribbon..... 10
Slika 3 WinMerge korisničko sučelje..... 12
Slika 4 Windows programski model 14
Slika 5 Hijerarhijsko stablo 'CObject' klase 17
Slika 6 Prikaz dokument/pogled veze 18
Slika 7 Aplikacijski čarobnjak..... 20
Slika 8 Resource editor..... 21
Slika 9 Sučelje aplikacije Uno..... 24
Slika 10 Izgled 'GameSettings' dijaloga 25
Slika 11 Dijaloški prozor otvoren AfxMessageBox() funkcijom..... 28

POPIS TABLICA

Tablica 1 Pregled verzija kroz godine 11
Tablica 2 Najčešće poruke..... 15
Tablica 3 Najčešće AFX funkcije..... 19

POPIS KODOVA

Kod 1 primjer switch naredbe unutar WndProc() 15
Kod 2 primjer mapiranja poruke 16
Kod 3 Primjer povezivanja klizača za unos količine igrača 25
Kod 4 Životni ciklus 'ChooseColorDlg' dijaloškog prozora 26
Kod 5 Funkcija koja stvara 'RoundWinDlg' dijaloški prozor 27
Kod 6 Prikaz poziva 'AfxMessageBox()' funkcije 27
Kod 7 Primjer korištenja 'AfxGetInstanceHandle()' funkcije 28
Kod 8 'OnDraw()' funkcija 29
Kod 9 Kod za stvaranje i ažuriranje 'playerListBox' 30
Kod 10 'OnMouseMove()' funkcija 31
Kod 11 'OnLButtonDown()' funkcija 32
Kod 12 Primjer 'SetTimer()' i 'OnTimer()' funkcija 33
Kod 13 'Message map' klase 'View' 34

1. UVOD

U suvremenom razvoju softvera, osobito u području izrade desktop aplikacija za Windows operacijski sustav, i dalje postoji velik broj rješenja temeljenih na tehnologijama koje su nastale prije modernih okvira poput .NET-a ili WinUI-a. Jedna od takvih tehnologija je Microsoft Foundation Classes (MFC), objektno-orijentirana C++ biblioteka koja pruža omotač (wrapper) oko Win32 API-ja i omogućuje izradu grafičkih korisničkih sučelja i upravljanje događajima na tradicionalan način. Unatoč svojoj starosti, MFC ostaje prisutan u brojnim industrijskim i naslijeđenim sustavima, gdje stabilnost, dugoročna održivost i kompatibilnost imaju prednost nad modernim vizualnim mogućnostima. Upravo zbog te trajnosti i praktične vrijednosti, MFC predstavlja zanimljiv predmet proučavanja, osobito u kontekstu analize razvijene aplikacije.

Tema će se obraditi kombinacijom teorijskog pregleda i praktične analize. Teorijski dio uključuje opis MFC biblioteke, njezinu arhitekturu i ključne koncepte. Praktični dio usmjeren je na analizu razvijene aplikacije: njezinu strukturu, glavne funkcionalne cjeline, način komunikacije između komponenti te korištenje MFC mehanizama poput dijaloga, kontrola i upravljanja porukama. Posebna pozornost posvećuje se dijelovima programa koji su najvažniji za razumijevanje njegove namjene i načina rada.

Struktura rada organizirana je tako da najprije daje teorijski okvir potreban za razumijevanje tehnologije, zatim prikazuje analizu aplikacije, a na kraju donosi zaključke o prednostima i nedostacima korištenja MFC-a u suvremenom razvoju softvera. Takav pristup omogućuje da se tema obradi cjelovito: od općeg pregleda tehnologije, preko konkretne implementacije, do kritičkog razmatranja njezine aktualnosti i mogućih smjerova daljnjeg razvoja.

2. PREGLED PODRUČJA

2.1 Povijest

Microsoft Foundation Classes (MFC) predstavlja skup C++ klasa namijenjenih razvoju aplikacija za operacijski sustav Windows. [1][2]

Prva verzija MFC biblioteke predstavljena je 1992. godine zajedno s razvojnim okruženjem Microsoft C/C++ 7.0. [3] Osnovna ideja bila je primijeniti principe objektno orijentiranog programiranja na razvoj Windows aplikacija. Umjesto korištenja velikog broja Win32 funkcija, programeri su mogli koristiti C++ klase koje su predstavljale prozore, dijaloge, gumbe i druge elemente korisničkog sučelja. Time je razvoj postao jednostavniji, pregledniji i lakši za održavanje.

U ranim godinama razvoja MFC je bio poznat pod nazivom Application Framework Extensions (AFX), zbog čega veliki broj MFC komponenti, poput funkcija i makroa, imaju naziv koji počinje prefiksom "Afx". Kasnije je biblioteka dobila današnji naziv Microsoft Foundation Classes, dok je prefiks ostao kao svojevrsni povijesni ostatak.

Slika 1 MFC logo



Izvor: [4]

Tijekom 1990-ih MFC je postao jedna od najvažnijih tehnologija za razvoj Windows desktop aplikacija. Njegova popularnost dodatno je porasla pojavom razvojnog okruženja Visual C++, koje je omogućilo grafičko stvaranje prozora, dijaloga i drugih komponenti aplikacije. Posebno važna bila je integracija s Visual Studio okruženjem, koje nudi čarobnjake (Wizards) za automatsko stvaranje početne strukture programa. Na taj način programeri su mogli razvijati složene aplikacije bez potrebe za ručnim pisanjem cjelokupnog temeljnog koda.

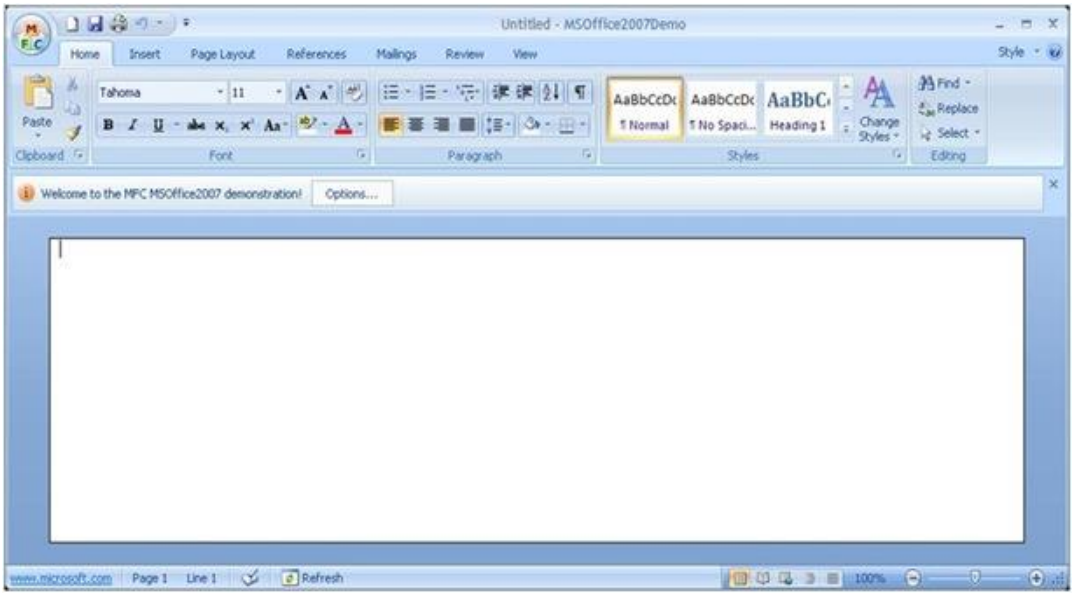
Značajan doprinos popularnosti MFC-a dala je i implementacija dokument/pogled arhitekture. Ovaj koncept omogućio je odvajanje podataka aplikacije od njihovog prikaza na zaslonu, čime su aplikacije postale modularnije i jednostavnije za održavanje. Takav pristup bio je posebno koristan kod poslovnih aplikacija, uređivača dokumenata i tehničkih alata.

Krajem 1990-ih i početkom 2000-tih MFC je dosegao vrhunac popularnosti. Velik broj komercijalnih i industrijskih aplikacija razvijan je upravo pomoću ove biblioteke. U tom razdoblju nastali su brojni razvojni alati, CAD sustavi, znanstvene aplikacije te poslovna rješenja koja su se oslanjala na MFC i Win32 platformu.

Početkom 21. stoljeća Microsoft je uveo .NET Framework, a kasnije i Windows Presentation Foundation (WPF). Te nove tehnologije donijele su moderniji pristup razvoju grafičkih korisničkih sučelja te su postupno preuzele ulogu glavnih razvojnih platformi za Windows aplikacije. Zbog toga je razvoj MFC-a usporen, ali biblioteka nije napuštena. Microsoft je nastavio pružati podršku te prilagođavati MFC novim verzijama Visual Studija i Windows operacijskog sustava.

Godine 2008. MFC je dobio značajnije proširenje koje je uključivalo podršku za moderne korisničke elemente kao što su ribbon sučelje, napredne alatne trake i poboljšane mogućnosti organizacije prozora. Cilj tih nadogradnji bio je omogućiti postojećim aplikacijama moderniji izgled bez potpune promjene tehnologije. [5]

Slika 2 Microsoft Office ribbon



Izvor: [5]

Danas se MFC smatra zrelom i stabilnom tehnologijom. Iako više nije prvi izbor za razvoj novih Windows aplikacija, i dalje je prisutan u velikom broju postojećih sustava. Razlozi za njegovu dugovječnost uključuju visoke performanse, izravnu povezanost s Win32 API-jem, stabilnost te činjenicu da mnoge organizacije održavaju aplikacije razvijene tijekom posljednjih nekoliko desetljeća. Microsoft i dalje uključuje MFC u

Visual Studio kao opcionalnu komponentu te omogućuje razvoj i održavanje postojećih projekata.

Tablica 1 Pregled verzija kroz godine

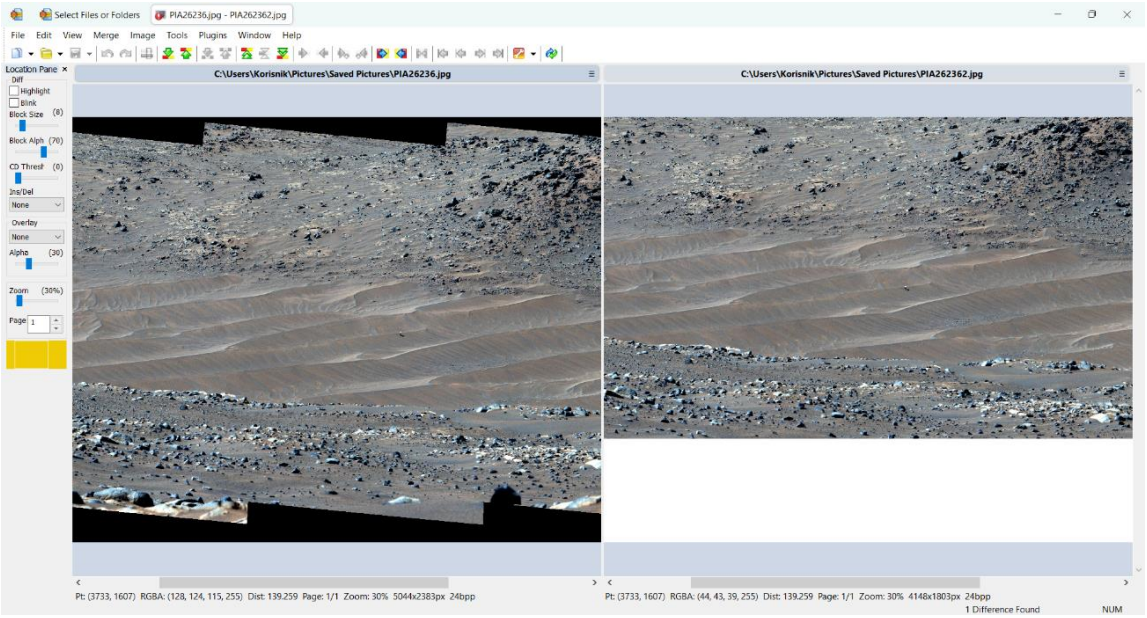
Godina	Verzija	Značajke
1992.	MFC 1.0	Prva verzija
1993.	MFC 2.0	Integracija s prvim verzijama Visual C++
1994.	MFC 3.x	Dodane nove klase i bolja integracija s 32-bitnim sustavima
1995.	MFC 4.0	Uvedena veća podrška za Win32 API
1996.	MFC 4.2	Dodana potpora za WinINet
1998.	MFC 6.0	Dolazi uz Visual Studio 6.0, MFC je u ovom periodu bio najpopularniji
2002.	MFC 7.0	Dolazi uz Visual Studio .NET
2003.	MFC 7.1	Dolazi uz Visual Studio .NET 2003
2005.	MFC 8.0	Dolazi uz Visual Studio 2005, poboljšana podrška za Windows sustave i sigurnosne mehanizme
2008.	MFC 9.0	Modernizacija sučelja, dodana podrška za ribbon, docking prozore te Office stil sučelja
2010. - danas	MFC 10.0 – MFC 14.x	Nema novih značajki, održava se kompatibilnost i sigurnosne nadogradnje. Uključen u Visual Studio 2026

Izvor: [1][3][6][7]

2.2 Usporedba

Prednost MFC-a u odnosu na druge okvire jest izravan pristup Win32 API-ju, što osigurava visoke performanse uz manje dodatnih apstrakcija. Zbog toga se i danas koristi u pojedinim industrijskim i alatnim aplikacijama, poput programa 'WinMerge' za uspoređivanje različitih dokumenata. [8]

Slika 3 WinMerge korisničko sučelje



Izvor: slikao autor

2.2.1 Microsoftovi aplikacijski okviri

Osnovna ideja novih Microsoftovih okvira poput WPF (Windows Presentation Foundation) i WinUI 3 za razliku od MFC-a je odmaknuti programere od Win32 modela i pojednostaviti izradu sučelja.

Za razliku od MFC-a, WPF i WinUI 3 koriste data binding i MVVM (Model-View-ViewModel) arhitekturni obrazac te XAML (Extensible Application Markup Language) temeljen na XML-u za definiranje sučelja.

Razlika između WPF i WinUI 3 je to što WPF koristi vlastiti sustav za prikaz korisničkog sučelja temeljen na DirectX-u, dok se WinUI 3 oslanja na Windows App SDK/WinRT. [9][10]

2.2.2 Nezavisne biblioteke i aplikacijski okviri

Po arhitekturi najbliža je wxWidgets biblioteka, izvorno razvijena 1992. godine pod imenom wxWindows te poput MFC-a radi kao C++ omotač za izvorne API-je

operacijskih sustava kao što su Windows, Linux i macOS uz ograničenu podršku za iOS. Primjer wxWidgets aplikacija je Code::Blocks IDE za C/C++, a biblioteka se koristi u industriji, znanosti i obrazovanju; među korisnicima su Xerox, AMD i NASA. [11]

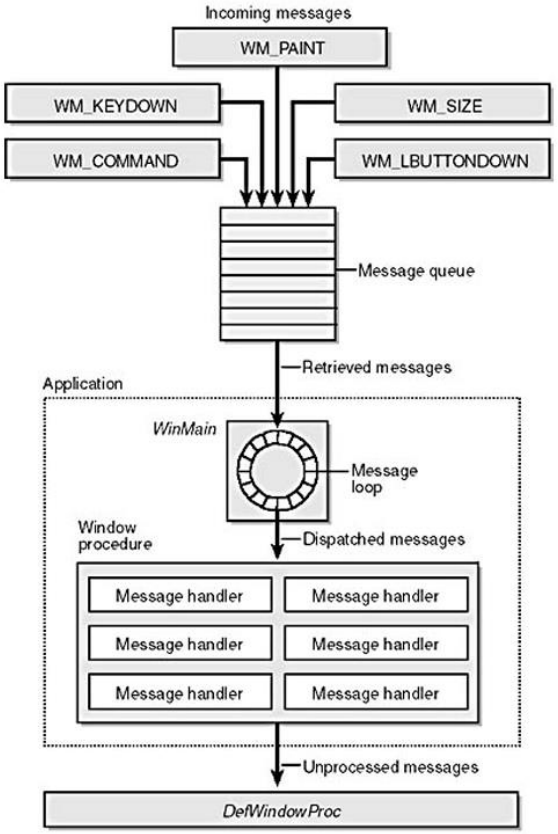
Još jedan C++ multiplatformski okvir vrijedan spomena je Qt koji pruža vrlo bogat skup biblioteka i moderno sučelje. U Qt-ju su napravljeni mnogi programi za uređivanje slika i medija, CAD te sučelja za embedded i mobilne sustave. [12]

3. FUNKCIONALNOST MFC-A

3.1 Windows programski model

Tradicionalni programi koriste proceduralni model izvođenja u kojem se naredbe izvršavaju redom od početka do kraja programa. Windows aplikacije koriste drugačiji pristup poznat kao event-driven programming model odnosno programski model temeljen na događajima. U takvom modelu aplikacija većinu vremena ne izvršava vlastiti programski kod, već čeka događaje koje generira operacijski sustav ili korisnik. Događaji mogu biti pritisak tipke na tipkovnici, klik mišem, promjena veličine prozora ili zahtjev za ponovnim iscrtavanjem sadržaja prozora. Kada se takav događaj dogodi, operacijski sustav generira odgovarajuću poruku i smješta je u red poruka (message queue). Program zatim preuzima poruke iz reda i prosljeđuje ih odgovarajućem prozoru na obradu. [2]

Slika 4 Windows programski model



Izvor: [2, str. 21]

Tablica 2 Najčešće poruke

Poruka	Događaj
WM_CHAR	Pritisnuta je tipka ili unesen znak
WM_COMMAND	Šalje se kada korisnik pozove stavku naredbe iz izbornika ili kada kontrola pošalje obavijesnu poruku svom roditeljskom prozoru
WM_LBUTTONDOWN	Pritisnut je lijevi gumb miša
WM_LBUTTONUP	Pušten je lijevi gumb miša
WM_MOUSEMOVE	Pomaknut je miš
WM_CREATE	Prozor je napravljen
WM_DESTROY	Prozor je uništen
WM_SIZE	Prozor je promijenio veličinu
WM_PAINT	Prozor se treba ponovno nacrtati
WM_QUIT	Aplikacija se terminira

Izvor: [2, str. 23-24]

U klasičnom Win32 programiranju obrada poruka odvija se u funkciji nazvanoj Window Procedure (WndProc). Ta funkcija sadrži switch naredbu koja za svaku poruku određuje odgovarajuće ponašanje. [1]

Kod 1 primjer switch naredbe unutar WndProc()

```
switch (message){
    case WM_PAINT:
            …
        break;
            …
    case WM_CLOSE:
            …
        break;
}
```

3.2 Mapiranje poruka (Message Maps)

Mapiranje poruka je jedan od važnijih mehanizama u MFC-u. Omogućava automatsko mapiranje Windows poruka i naredbi na metode u klasama bez ručnog pozivanja 'RegisterClass()' i switch naredbe unutar 'WndProc()' funkcije (Kod 1). [1][2]

Kod 2 primjer mapiranja poruke

```
// MyView.h
class CMyView : public CView {
    afx_msg void OnFileOpen();
    DECLARE_MESSAGE_MAP()
};

// MyView.cpp
BEGIN_MESSAGE_MAP(CMyView, CView)
    ON_COMMAND(ID_FILE_OPEN, &CMyView::OnFileOpen)
END_MESSAGE_MAP()

void CMyView::OnFileOpen() {...}
```

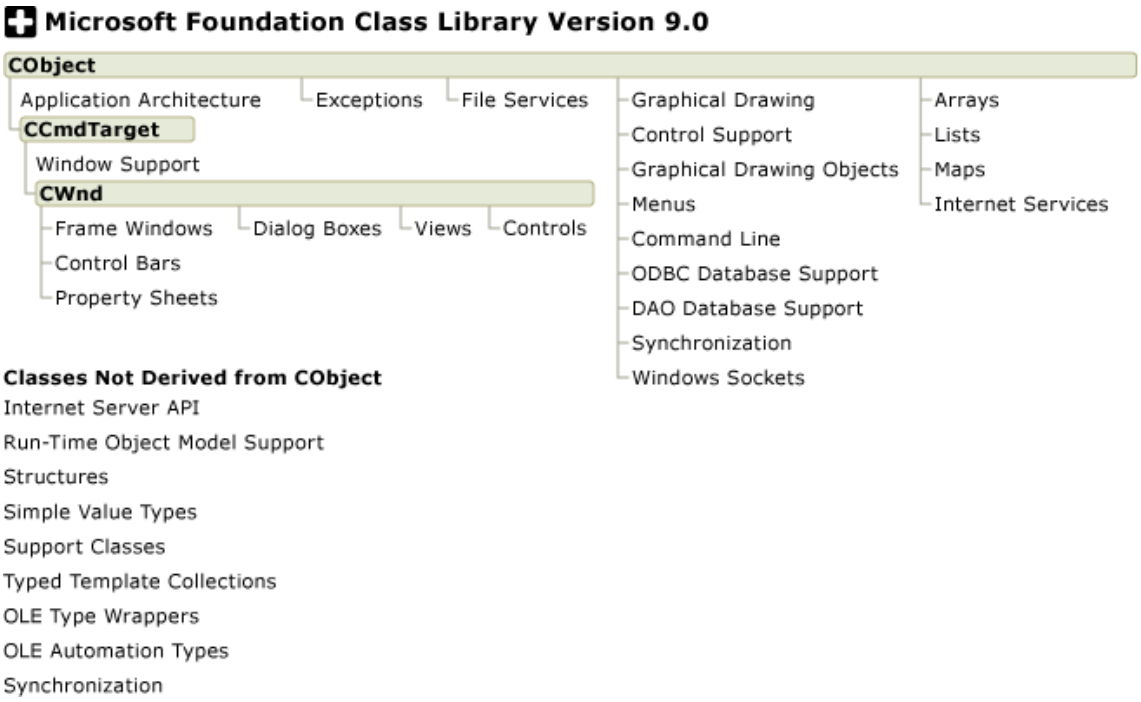
U Kodu 1 prikazan je tradicionalni Win32 pristup obradi poruka. Sve poruke obrađuju se unutar funkcije 'WndProc()' pomoću velike switch naredbe. Kako aplikacija raste, broj podržanih poruka postaje sve veći, zbog čega funkcija postaje nepregledna i teža za održavanje. Nasuprot tome, Kod 2 prikazuje MFC pristup korištenjem mehanizma mapiranja poruka. Umjesto ručnog provjeravanja identifikatora poruke unutar switch naredbe, MFC kroz makroe 'BEGIN_MESSAGE_MAP()' i 'END_MESSAGE_MAP()' automatski povezuje poruku ili naredbu s odgovarajućom metodom klase.

U prikazanom primjeru naredba identificirana vrijednošću 'ID_FILE_OPEN' povezana je s metodom 'OnFileOpen()'. Kada korisnik aktivira naredbu, MFC automatski poziva odgovarajuću metodu bez potrebe za ručnim prepoznavanjem poruke unutar funkcije 'WndProc()'.

3.3 Hijerarhija klasa MFC-a

MFC je organiziran kao velika hijerarhija međusobno povezanih C++ klasa. Većina klasa unutar biblioteke izravno ili neizravno nasljeđuje klasu 'CObject', koja predstavlja temelj gotovo cijelog MFC-a. [2]

Slika 5 Hijerarhijsko stablo 'CObject' klase



Izvor: [13]

Najvažnije funkcionalnosti klase 'CObject' su podrška za serijalizaciju (serialization), Run-Time Class Information (RTCI) i podrška za dijagnostiku i otklanjanje pogrešaka. Serijalizacija predstavlja postupak spremanja podataka objekta na trajni medij, primjerice datoteku na disku, te njihovo kasnije ponovno učitavanje. Primjer primjene serijalizacije uključuje:

- spremanje dokumenata
- spremanje postavki aplikacije
- spremanje konfiguracijskih podataka

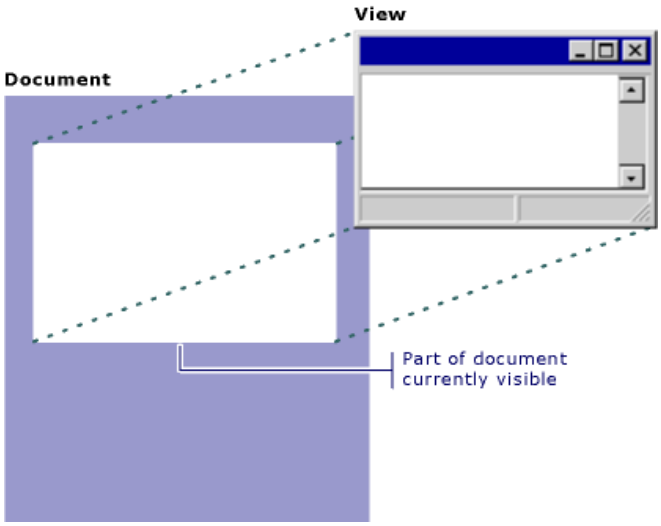
Ova funkcionalnost posebno je važna u aplikacijama koje koriste dokument/pogled arhitekturu.

RTCI mehanizam omogućuje određivanje tipa objekta tijekom izvođenja programa. Primjerice, aplikacija može provjeriti pripada li neki objekt određenoj klasi bez potrebe za ručnim vođenjem informacija o tipu objekta. MFC je implementirao vlastiti sustav RTCI-ja prije nego što je standard C++ uveo mehanizam poznat kao Run-Time Type Information (RTTI). [2]

3.4 Dokument/pogled arhitektura

Jedna od važnijih sastavnica MFC aplikacijskog okvira je dokument/pogled arhitektura. Ovaj model predstavlja temelj mnogih MFC aplikacija jer omogućuje odvajanje podataka aplikacije od njihovog prikaza na korisničkom sučelju. MFC podršku za ovu arhitekturu implementira kroz klase 'CDocument', 'CView', 'CFrameWnd' i 'CDocTemplate' te druge komponente koje povezuju pojedine dijelove aplikacije u funkcionalnu cjelinu. Takvo odvajanje omogućuje da se isti podaci prikazuju na više načina bez potrebe za dupliciranjem podataka. Na primjer, jedna aplikacija može imati više prikaza istog dokumenta, pri čemu svaka instanca klase 'CView' prikazuje sadržaj na drugačiji način. Primjeri uključuju: spremanje i učitavanje dokumenata, ispis i pregled ispisa (Print Preview), podršku za višestruke prikaze istog dokumenta te integraciju s OLE (Object Linking and Embedding) i Active Document tehnologijama. [2][14]

Slika 6 Prikaz dokument/pogled veze



Izvor: [14]

3.5 AFX funkcije

Osim velikog broja C++ klasa, MFC sadrži i skup globalnih funkcija čiji nazivi započinju prefiksom Afx. Za razliku od članskih funkcija koje pripadaju određenim objektima, AFX funkcije mogu se pozivati iz bilo kojeg dijela programa bez potrebe za pristupom konkretnom MFC objektu. Ove funkcije predstavljaju dodatni sloj apstrakcije iznad Win32 API-ja te programeru omogućuju jednostavniji pristup funkcionalnostima okruženja. [2]

Tablica 3 Najčešće AFX funkcije

Funkcija	Namjena
AfxMessageBox()	Prikaz dijaloškog okvira s porukom
AfxGetApp()	Dohvaća pokazivač (pointer) na aplikaciju
AfxGetAppName()	Dohvaća naziv aplikacije
AfxGetMainWnd()	Dohvaća pokazivač na glavni prozor aplikacije
AfxGetInstanceHandle()	Dohvaća instance handle trenutne aplikacije
AfxBeginThread()	Započinje nit (thread)
AfxEndThread()	Završava nit
AfxAbort()	Prekida izvođenje aplikacije

Izvor: [2, str.46-47]

4. PODRŠKA U VISUAL STUDIJU

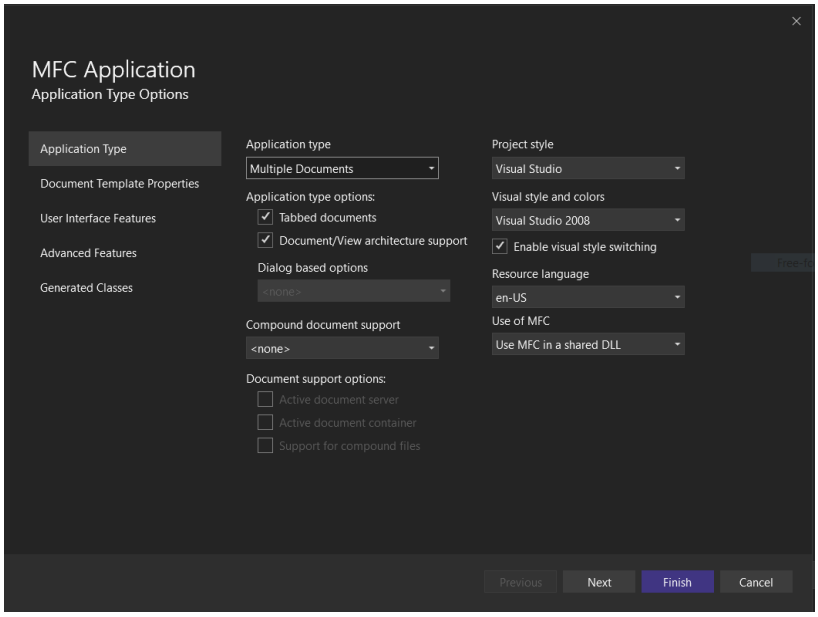
4.1 Integracija MFC-a u Visual Studio

MFC je od samih početaka usko povezan s Microsoftovim razvojnim okruženjem Visual C++. Programerima je omogućeno stvaranje, uređivanje, kompajliranje i otklanjanje pogrešaka MFC aplikacija unutar jedinstvenog razvojnog okruženja. Prilikom instalacije Visual Studija moguće je odabrati komponentu „C++ MFC for x64/x86 (Latest MSVC)“ koja uključuje biblioteke i zaglavlja za razvoj MFC i ATL (Active Template Library) aplikacija. MFC je i dalje podržan u aktualnim verzijama Visual Studija, što omogućuje održavanje postojećih aplikacija, ali i razvoj novih Windows desktop programa temeljenih na ovoj tehnologiji. [15]

4.2 MFC aplikacijski čarobnjak

Jedna od važnijih značajki Visual Studija za MFC je aplikacijski čarobnjak. Čarobnjak omogućuje odabir vrste aplikacije (SDI, MDI, dijaloška aplikacija), [15] izgleda sučelja (ribbon ili klasično), podrške za arhitekturu dokument/pogled te za ActiveX i COM.

Slika 7 Aplikacijski čarobnjak



Izvor: slikao autor

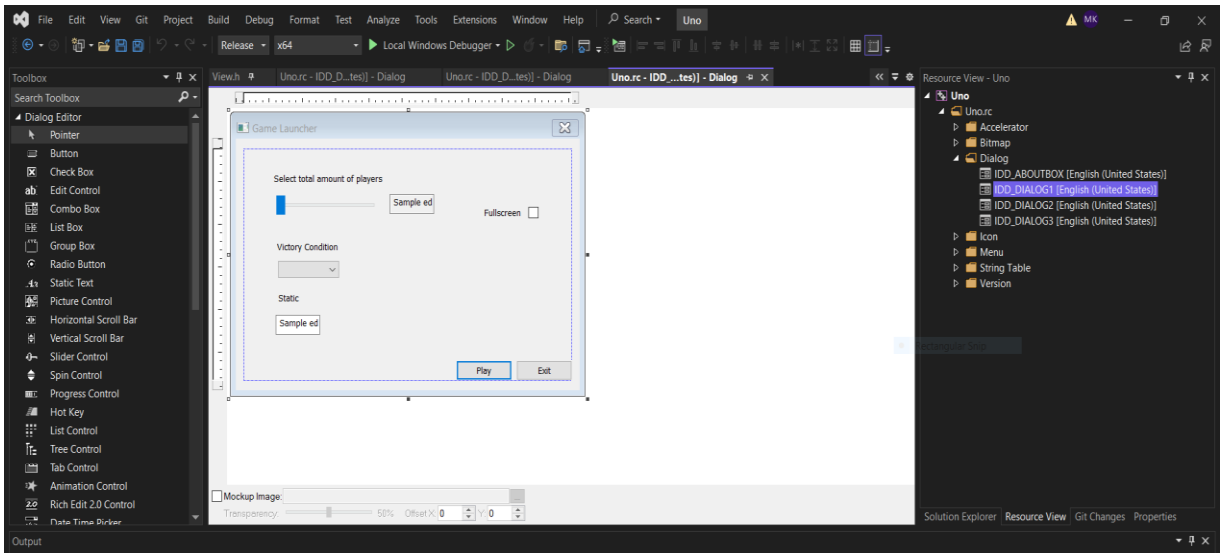
4.3 Uređivači resursa

Visual Studio uključuje Resource Editor, alat za vizualno oblikovanje korisničkog sučelja. [15] S pomoću njega moguće je izrađivati i uređivati:

- dijaloške okvire,
- izbornike,
- alatne trake,
- ribbon sučelja,
- ikone,
- pokazivače miša,
- string tablice.

Elementi se pohranjuju u resursne datoteke (.rc) koje Visual Studio automatski povezuje s aplikacijom.

Slika 8 Resource editor



Izvor: slikao autor

5. PRAKTIČAN RAD – UNO KARTAŠKA IGRA

5.1 Svrha aplikacije

Aplikacija je računalna verzija poznate kartaške igre Uno.

Pravila igre:

- Igra se protiv 1 do 9 računalnih igrača.
- Prije početka igre korisnik odabire broj igrača i trajanje igre. Moguće je odabrati jednu partiju, igru do određenog broja pobjeda ili tradicionalno igranje na bodove.
- Na +2 i +4 je moguće odgovoriti s +2 odnosno +4, nakon čega sljedeći igrač mora odgovoriti ili vući zbroj svih +2 ili +4 karata. Igrač koji je vukao zbog +2 ili +4 ne gubi potez i, ako ne može odigrati kartu, vuče još jednu.
- U igri s jednim protivnikom, nakon odigravanja karte za promjenu smjera ili preskakanja, igrač ponovno dolazi na red i mora vući kartu ako nema druge karte za odigrati, osim ako mu je to bila posljednja karta.

5.2 Struktura aplikacije

Aplikacija je izrađena korištenjem MFC biblioteke i programskog jezika C++. Za prikaz karata i korisničkog sučelja korišten je GDI+, dok su dijaloški okviri i standardne kontrole implementirani korištenjem MFC klasa.

Klasa 'CUnoGameApp' je izvedena iz 'CWinApp'. Predstavlja ulaznu točku aplikacije i odgovorna je za:

- inicijalizaciju aplikacije, glavnog prozora i game objekta,
- pokretanje GDI+ sustava.

Klasa 'CMainFrame' je izvedena iz 'CFrameWnd' i predstavlja glavni prozor. Može se proširiti preko cijelog ekrana ili smanjiti na minimalnu širinu od 800 i visinu od 600 piksela. Za razliku od klasične dokument/pogled arhitekture, 'CMainFrame' stvara 'View' i umjesto 'CDocument' i 'CDocTemplate' klasa, aplikacija koristi vlastitu 'Game' klasu za pojednostavljenu prozor + pogled arhitekturu.

Klasa 'View' je izvedena iz 'CView' i predstavlja glavno sučelje (Slika 9). Ova klasa je odgovorna za:

- prikaz slika svih karata u ruci igrača s uvećanim slikama karata koje korisnik može odigrati, uvećanu sliku karte u ruci ako je korisnik pozicionirao kursor miša iznad te karte i zadnju kartu u špilu odigranih karata,
- korisnički unos odigravanja karata ili gumba,
- prikaz stanja svih igrača,
- komunikaciju s modelom igre,
- otvaranje prozora za promjene boje, upozorenja o kršenju pravila kada igrač ne stisne gumb 'Uno' kada je potrebno i obavijest za kraj partije ili igre.

Klasa 'Scoreboard' služi za bilježenje bodova svakog igrača i nakon svake partije vraća informaciju je li igrač pobijedio igru.

Struktura 'SavedGameSettings' služi za spremanje podataka koje je korisnik unio pri početku igre.

Klasa 'Game' predstavlja središnji dio aplikacije i sadrži kompletnu logiku igre:

- tijek igre,
- promjenu igrača,
- provjeru valjanosti poteza,
- obračun bodova,
- određivanje pobjednika.

Klasa 'Player' predstavlja pojedinog igrača i sadrži algoritme koji služe za logiku računalnih igrača. Algoritam preferira karte višeg tipa iz boje koje najviše posjeduje. U slučaju korištenja karte za promjenu boje, odabire boju koju najviše posjeduje.

Klasa 'Hand' služi za:

- sortiranje karata,
- provjeru mogućih poteza,
- uklanjanje odigrane karte.

Klasa 'Deck' predstavlja špil karata i sadrži metode za:

- miješanje karata,
- izvlačenje karata,
- ponovno stvaranje špila u početku partije ili uzimanje iz špila odigranih karata kada se odigraju sve karte.

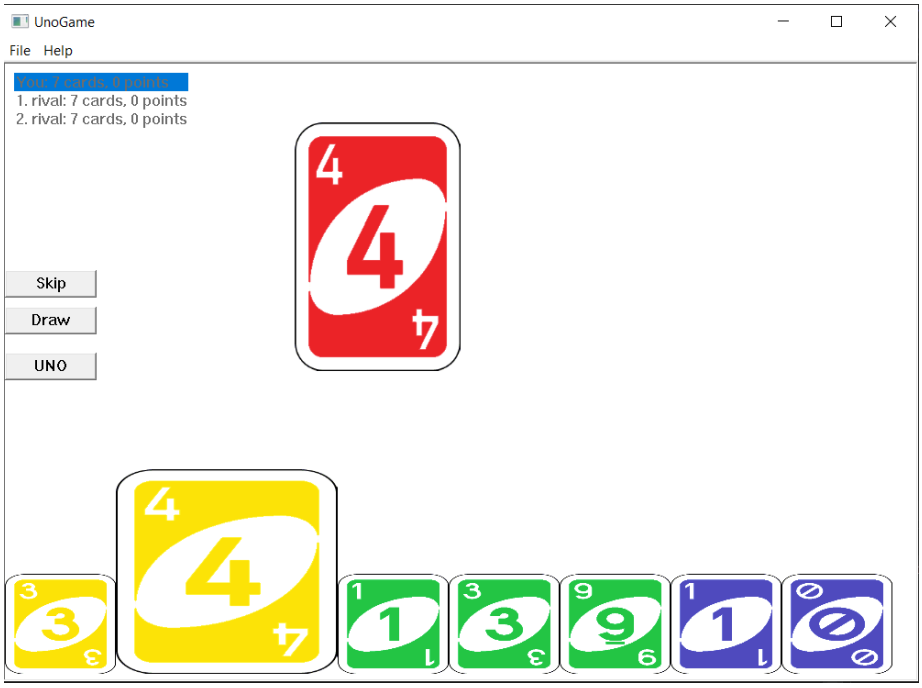
Klasa 'PlayedCards' predstavlja špil odigranih karata.

Klasa 'Card' predstavlja osnovni podatkovni objekt igre. Definirana je koristeći enumeracije koje opisuju:

- boju karte (crvena, žuta, zelena, plava, džoker),
- tip karte (nula, jedan, dva, tri, četiri, pet, šest, sedam, osam, devet, preskakivanje, promjena smjera, vuci dvije, promjena boje, vuci četiri).

Svaka karta ima svoj ID koji se računa formulom: boja karte*100 + tip karte što daje numeričku vrijednost koja predstavlja ID bitmape koju je potrebno prikazati. Enumeracija boja počinje od vrijednosti 50, a tipa od 0. Na primjer, karta „plava devetka“ ima vrijednost 53*100+9=5309, a bitmapa 'IDB_BLUE_9' definirana je s vrijednošću 5309.

Slika 9 Sučelje aplikacije Uno



Izvor: slikao autor, [16]

5.3 Korištene MFC komponente

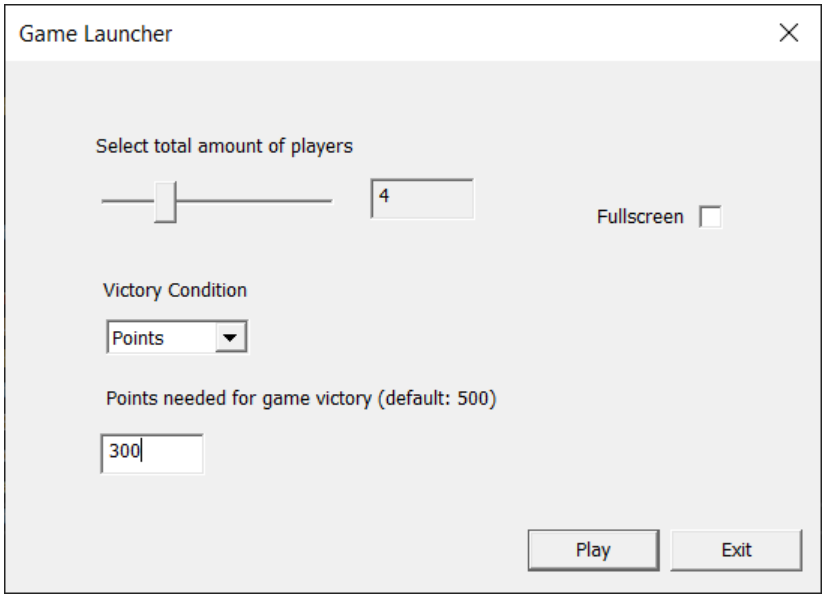
5.3.1 Dijaloški prozori

Klasa 'GameSettings' je izvedena iz 'CDialogEx' klase. Prikazuje se pri pokretanju aplikacije i sadrži korisničke unose:

- 'CButton' u obliku kvačice za prikaz igre preko cijelog ekrana,

- 'CSliderCtrl' klizač s vrijednostima od 2 do 10 i 'CEdit' koji prikazuje vrijednost unosa. Budući da je korisnik i sam igrač, vrijednosti označavaju količinu računalnih protivnika od 1 do 9,
- 'CComboBox' za odabir trajanja igre, 'CStatic' tekst za opis odabira te 'CEdit' za ručni unos.

Slika 10 Izgled 'GameSettings' dijaloga



Izvor: slikao autor

MFC koristi DDX (Dialog Data Exchange) mehanizam za povezivanje kontrola definiranih u resursnim datotekama s objektima i varijablama unutar programa. U aplikaciji Uno koristi se funkcija 'DDX_Control', kojom se kontrole poput 'CSliderCtrl', 'CEdit', 'CComboBox' i 'CButton' povezuju s odgovarajućim članovima klase 'GameSettings'. Time je omogućen jednostavan pristup kontrolama kroz objektno-orijentirano sučelje bez ručnog dohvaćanja Windows handleova.

Kod 3 Primjer povezivanja klizača za unos količine igrača

```
void GameSettings::DoDataExchange(CDataExchange* pDX){
	DDX_Control(pDX, IDC_SLIDER1, PlayerCountSlider);
	PlayerCountSlider.SetRangeMin(2);
	PlayerCountSlider.SetRangeMax(10);
	...
}
void GameSettings::OnBnClickedOk(){
	settings->playerCount = PlayerCountSlider.GetPos();
	...
}
```

Klasa 'ChooseColorDlg' je izvedena iz 'CDialogEx' i služi za odabir promjene boje kada korisnik odigra jednu od crnih džoker (wildcard) karata.

Kod 4 Životni ciklus 'ChooseColorDlg' dijaloškog prozora

```
//View.cpp
switch (game->validatePlayerMove(m_hoveredImageIndex)){
...
case 1: {
    ChooseColorDlg chooseDlg;
    if (chooseDlg.DoModal() == IDOK) {
        int color = chooseDlg.getChosenColor();
            if (game->PlayerMove(m_hoveredImageIndex, color)) {
        ...
            }
        ...
}}}
//ChoseColorDlg.cpp
BEGIN_MESSAGE_MAP(ChooseColorDlg, CDialogEx)
      ON_BN_CLICKED(IDC_RED, &ChooseColorDlg::OnBnClickedButton1)
      ...
END_MESSAGE_MAP()
void ChooseColorDlg::OnBnClickedButton1(){
      chosenColor = IDC_RED;
      this->EndDialog(IDOK);
}
//Game.cpp
bool Game::PlayerMove(const int i, int color){
      ...
      if (i != -1) {
            Card card = getPlayerhand()[i];
            switch (card.getType()) {
            ...
            case ColorChange: {
                  card.Color = (CardColors)color;
                  playedCards.playerTurn(card);
                  getPlayerhand().erase(getPlayerhand().begin() + i);
                  ...
                  }
            ...
            }}
      ...
}
```

Klasa 'RoundWinDlg' je izvedena iz 'CDialogEx' i pojavljuje se kada neki igrač pobijedi partiju. Korisnik će biti obaviješten koji je to igrač bio te može odabrati hoće li nastaviti igru ili izaći iz aplikacije. Također ako je neki od igrača završio igru pobjedom na bodove, korisnik je isto obaviješten o tome.

Kod 5 Funkcija koja stvara 'RoundWinDlg' dijaloški prozor

```
void View::ShowWinScreen(int player, const bool rndWin){
    RoundWinDlg rwDlg(player,rndWin,nullptr);
    if (rwDlg.DoModal() == IDOK) {
        game.resetGame(rndWin);
        Invalidate();
        UpdateWindow();
        UpdateListBox();
        return;
    }
    else {
        AfxGetMainWnd()->PostMessage(WM_CLOSE);
    }
}
```

Dijaloški prozor 'About' korisnik otvara na izborničkoj traci pod opcijom 'Help' i sadrži tekst koji navodi kućna pravila (stacking, preskakanje, UNO) i autora.

5.3.2 Afx funkcije

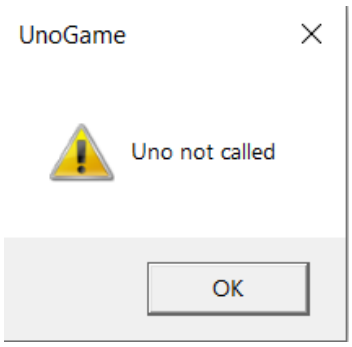
Afx funkcije koje se koriste u programu uključuju 'AfxMessageBox()', 'AfxGetMainWnd()' i 'AfxGetInstanceHandle()'.

'AfxMessageBox()' otvara dijaloški prozor. Primjer u aplikaciji Uno je upozorenje kada korisnik prekrši pravila Una i odigra pretposljednju kartu bez prethodnog pritiska na gumb Uno.

Kod 6 Prikaz poziva 'AfxMessageBox()' funkcije

```
if(!isUno && game.getPlayerhand().size() <= 1) {
      char UnoWarning[16];
      ::LoadString(0, 109, UnoWarning, sizeof UnoWarning);
      AfxMessageBox(UnoWarning);
      game.PlayerUNOdraw();
      Invalidate();
      UpdateWindow();
}
```

Slika 11 Dijaloški prozor otvoren AfxMessageBox() funkcijom



Izvor: slikao autor

'AfxGetMainWnd()' funkcija dohvaća pokazivač na glavni prozor, 'CMainFrame'. Primjer njezine uporabe prikazan je u Kodu 5, gdje ako korisnik odluči ne nastaviti igru. 'AfxGetMainWnd()' se poziva i šalje glavnom prozoru poruku za zatvaranje.

„`AfxGetMainWnd()->PostMessage(WM_CLOSE);`“

'AfxGetInstanceHandle()' funkcija dohvaća 'handle instance' (HINSTANCE) ili 'handle module' (HMODULE) poput EXE ili DLL koji služe kao identifikator za modul u memoriji. Neophodne su za funkcije koje upravljaju resursima. Primjer u aplikaciji je za crtanje zadnje karte u špilu odigranih karata.

Kod 7 Primjer korištenja 'AfxGetInstanceHandle()' funkcije

```
//View.cpp
void View::ShowPlayedCard(CDC* pDC,const Card card) const {
    HMODULE hModule = AfxGetInstanceHandle();
    Gdiplus::Bitmap* pImage=
      Gdiplus::Bitmap::FromResource(hModule,
      MAKEINTRESOURCEW(card.getColor() * 100 + card.getType()));
      ...
}
```

5.3.3 Dvostruko međuspremanje (Double Buffering)

Prilikom crtanja grafičkog sadržaja izravno na zaslon svaka pojedinačna operacija crtanja odmah postaje vidljiva korisniku. Ako se na zaslonu mora nacrtati veći broj objekata, primjerice karte, korisnik može primijetiti kratkotrajno titranje slike (flickering). Problem nastaje zato što se tijekom crtanja nakratko vide međufaze procesa iscrtavanja. Dodatni problem pojavljuje se kod čestog osvježavanja sadržaja, primjerice pri prikazu uvećanog pregleda karte ili tijekom poteza računalnih igrača. Dvostruko međuspremanje rješava taj problem korištenjem posebnog memorijskog spremnika

(*back buffer*). Umjesto izravnog crtanja na zaslon, kompletan sadržaj prvo se crta u memoriji, a tek nakon završetka crtanja gotova slika prenosi se na zaslon. Primjer je „`View::OnDraw(CDC* pDC)`“ metoda koja se ne poziva direktno, nego je poziva MFC kada Windows pošalje poruku za ponovno iscrtavanje prozora (WM_PAINT). To može biti:

- prilikom prvog stvaranja prozora ili promjene veličine prozora,
- pozivanje funkcija 'Invalidate()' ili 'InvalidateRect()', koje označavaju cijeli prozor ili dio prozora kao nevažeći sadržaj koji treba ponovno iscrtati,
- pozivanje funkcije UpdateWindow(), koja odmah uzrokuje obradu poruke WM_PAINT ako nevažeće područje već postoji.

Kod 8 'OnDraw()' funkcija

```
void View::OnDraw(CDC* pDC){
    CRect clientRect;
    GetClientRect(&clientRect);
    CDC memoryDC;
    memoryDC.CreateCompatibleDC(pDC);
    CBitmap bitmap;
    bitmap.CreateCompatibleBitmap(pDC, clientRect.Width(),
      clientRect.Height());
    CBitmap* pOldBitmap = memoryDC.SelectObject(&bitmap);
    memoryDC.FillSolidRect(&clientRect, RGB(255, 255, 255));
    Graphics graphics(memoryDC.GetSafeHdc());
    ShowPlayedCard(&memoryDC, game->playedCards.getLast());
    if (!hand_bitmaps.empty()){
        ShowHand(&memoryDC);
        if (m_hoveredImageIndex >= 0 &&
            m_hoveredImageIndex<hand_bitmaps.size())
        {
            ShowPreview(&memoryDC, hand_bitmaps[m_hoveredImageIndex]);
        }
    }
    pDC->BitBlt(0, 0, clientRect.Width(), clientRect.Height(),
&memoryDC, 0, 0, SRCCOPY);
    memoryDC.SelectObject(pOldBitmap);
    DeleteObject(pOldBitmap);
    DeleteDC(memoryDC);
}
```

5.3.4 Ostale komponente korištene u 'View' klasi

'View' sadrži kontrole za:

- 'CButton' gumbe 'Draw', 'Skip' i 'Uno'. Gumb 'Draw' koristi se za izvlačenje jedne ili više karata iz špila, uključujući situacije kada je prethodni igrač odigrao kartu +2 ili +4. Gumb 'Skip' omogućuje preskakanje poteza kada ne postoji valjan potez. Gumb 'Uno' koristi se za prijavu stanja UNO prije odigravanja pretposljednje karte.
- 'CListBox' kontrolu 'playerListBox' za prikaz stanja svih igrača. Korisnik ne može mijenjati odabir unutar liste jer je kontrola stvorena sa stilom WS_DISABLED, dok se pokazivač (selektirani redak) koristi za označavanje igrača koji je trenutačno na potezu.

Kod 9 Kod za stvaranje i ažuriranje 'playerListBox'

```
//View.cpp
afx_msg int View::OnCreate(LPCREATESTRUCT lpCreateStruct) {
    ...
    playerListBox.Create(WS_CHILD | WS_VISIBLE | WS_VSCROLL |
LBS_HASSTRINGS | WS_DISABLED, rect, this, NULL);
    UpdateListBox();
      ...
}
void View::UpdateListBox(){
    char Player[32];
    ::LoadString(0, 110, Player, sizeof Player);
    playerListBox.ResetContent();
    CString str;
    str.Format(Player, game->getPlayerhand().size(),
      game->scBoard.GetPlayerPoints(0));
    playerListBox.AddString(str);
    for (UINT i = 1; i < game->playerCount; ++i) {
        ::LoadString(0, 111, Player, sizeof Player);
        str.Format(Player, i, game->players[i].playerHand.hand.size(),
      game->scBoard.GetPlayerPoints(i));
        playerListBox.AddString(str);
    }
    playerListBox.SetCurSel(game->currentPlayer);
}
```

'View' klasa također mapira funkcije poput 'OnMouseMove()', 'OnLButtonDown()' i 'OnTimer()'. 'OnMouseMove()' se poziva kada se pomakne miš, a kada je miš na nekoj

karti pokazuje se uvećani pregled te karte i pohranjuje se indeks te karte. Zatim lijevim pritiskom miša ('OnLButtonDown()') odigrava se karta spremljena u indeks, to jest ako se pokazivač i dalje nalazi na karti koja se može odigrati.

Kod 10 'OnMouseMove()' funkcija

```
void View::OnMouseMove(UINT nFlags, CPoint point) {
    bool hoverChanged = false;
    CRect hand;
    GetHandRect(hand);
    if (hand.PtInRect(point)) {
        for (int i = 0; i < m_imageRects.size(); ++i){
            if (m_imageRects[i].PtInRect(point)) {
                if (m_hoveredImageIndex != i){
                    hoverChanged = true;
                    CRect previewRect;
                    GetPreviewRect(previewRect);
                    InvalidateRect(previewRect, FALSE);
                    m_hoveredImageIndex = i;
                }
                break;
            }
        }
    }
    if (!hoverChanged && m_hoveredImageIndex != -1 &&
        !m_imageRects[m_hoveredImageIndex].PtInRect(point)){
        hoverChanged = true;
        CRect previewRect;
        GetPreviewRect(previewRect);
        InvalidateRect(previewRect, FALSE);
        m_hoveredImageIndex = -1;
    }
    if (hoverChanged){
        UpdateWindow();  // Force the view to be redrawn
    }
    CView::OnMouseMove(nFlags, point);
}
```

Kod 11 'OnLButtonDown()' funkcija

```
afx_msg void View::OnLButtonDown(UINT nFlags, CPoint point) {
    if (m_hoveredImageIndex > -1 &&
        m_hoveredImageIndex < hand_bitmaps.size() &&
        game->currentPlayer == 0){
        switch (game->validatePlayerMove(m_hoveredImageIndex)) {
        case -1: {
            break;
        }
        case 0: {
            ...
            break;
            }
        case 1: {
            ...
            }
        }
        ...
        if (game->currentPlayer != 0) {
            UpdateListBox();
            ::SetTimer(*this, MyTimerId, 1000, NULL);
        }
    }
}
```

Funkcija 'OnTimer()' omogućava događaje koji se obavljaju kada istekne tajmer. U Uno aplikaciji omogućuje korisniku ljudski vidljive poteze svih računalnih igrača tako što se svaki potez odigra nakon intervala tajmera.

Kod 12 Primjer 'SetTimer()' i 'OnTimer()' funkcija

```
//funkcije koje pozivaju tajmer
afx_msg void View::OnLButtonDown(UINT nFlags, CPoint point) {
      ...
      ::SetTimer(*this,       //handle na glavni prozor
       MyTimerId,             //identifikator tajmera
      1000,                   //trajanje intervala
      NULL);                  //callback funkcija
}

void View::OnSkipButtonClick(){
    ...
    ::SetTimer(*this, MyTimerId, 1000, NULL);
}



//funkcija koja se poziva nakon intervala tajmera
void View::OnTimer(UINT_PTR timerId){
    if(game->currentPlayer != 0) {
        if (game->processMove()) {
            KillTimer(timerId);
            ShowWinScreen(game->currentPlayer, game->UpdatePoints());
            UpdateListBox();
            Invalidate();
            UpdateWindow();
            return;
        }
        UpdateListBox();
        Invalidate();
        UpdateWindow();
    }
    else {
        KillTimer(timerId);
    }
    CView::OnTimer(timerId);
}
```

'View' koristi vlastitu 'message mapu' za povezivanje Windows poruka i događaja korisničkog sučelja s odgovarajućim metodama klase. Koristi se za:

- obradu stvaranja prikaza (ON_WM_CREATE),
- obradu klikova na gumbe (ON_BN_CLICKED),

- odabir karata klikom miša (ON_WM_LBUTTONDOWN),
- prikaz povećanog pregleda karte tijekom pomicanja pokazivača miša (ON_WM_MOUSEMOVE),
- izvođenje poteza računalnih igrača korištenjem tajmera (ON_WM_TIMER).

Kod 13 'Message map' klase 'View'

```
IMPLEMENT_DYNCREATE(View, CView)
BEGIN_MESSAGE_MAP(View, CView)
    ON_WM_CREATE()
    ON_BN_CLICKED(1001, &View::OnDrawButtonClick)
    ON_BN_CLICKED(1002, &View::OnSkipButtonClick)
    ON_BN_CLICKED(1003, &View::OnUnoButtonClick)
    ON_WM_LBUTTONDOWN()
    ON_WM_MOUSEMOVE()
    ON_WM_TIMER()
END_MESSAGE_MAP()
```

5.4 Korištenje GDI+ biblioteke

GDI+ (Graphics Device Interface Plus) predstavlja proširenje klasičnog Windows grafičkog sustava GDI. [17] GDI+ omogućuje:

- prikaz bitmap slika,
- transparentnost (alpha channel),
- anti-aliasing,
- gradijente,
- transformacije i skaliranje slika,
- podršku za formate JPEG, PNG, BMP i GIF.

GDI+ omogućuje kvalitetnije i jednostavnije crtanje u Win32 okruženju, ali se danas smatra zastarjelim zbog:

- oslanjanja na procesor (CPU) za izvođenje većine grafičkih operacija,
- slabijih performansi u grafički zahtjevnijim scenarijima,
- ograničene podrške za moderne animacije i efekte,
- nedostatka podrške za 3D prikaz.

6. ZAKLJUČAK

Promatrajući MFC iz današnje perspektive, više nije suvremeno rješenje za razvoj novih desktop aplikacija. Mnogi problemi koje je MFC uspješno rješavao početkom 1990-ih danas se rješavaju drugačijim pristupima temeljenima na data bindingu, MVVM obrascu i deklarativnom razvoju korisničkog sučelja u okvirima poput WPF-a i WinUI-ja. Nedostatak ugrađene podrške za podatkovno povezivanje, oslanjanje na mehanizam mapiranja poruka te veća količina programskog koda potrebna za izradu korisničkog sučelja čine razvoj sporijim u usporedbi s modernim okvirima.

S druge strane, MFC nije zastario u smislu da više nema praktičnu primjenu. Njegova povezanost s Win32 API-jem omogućuje vrlo dobre performanse i veliku razinu kontrole nad operacijskim sustavom. Upravo zbog toga i dalje postoji značajan broj aplikacija koje se oslanjaju na MFC ili izravno na Win32. Međutim, kod novih projekata u kojima su performanse jednako važne često se koriste alternativni okviri poput Qt-a ili wxWidgets-a, koji nude slične mogućnosti uz veću podršku za više platformi.

Razvoj praktičnog dijela rada dodatno je pokazao način na koji se teorijski koncepti MFC-a primjenjuju u stvarnim aplikacijama. Razvijanjem UNO kartaške igre potvrđeno je da MFC i dalje omogućuje razvoj funkcionalnih desktop aplikacija koje objedinjuju grafičko korisničko sučelje, obradu događaja i logiku aplikacije. Korištenje dijaloških prozora, GDI+ biblioteke, Message Maps mehanizma i različitih MFC kontrola omogućilo je razumijevanje unutarnjeg rada MFC aplikacija te prednosti koje ovakav pristup donosi u organizaciji programskog koda.

Može se zaključiti da MFC, unatoč svojoj starosti, i dalje predstavlja važan dio Windows razvojnog ekosustava iako nije prvi izbor za razvoj modernih korisničkih sučelja. MFC stoga zadržava značajnu obrazovnu i praktičnu vrijednost te predstavlja važnu tehnologiju u povijesti razvoja aplikacija za Windows operacijski sustav.

LITERATURA

1. Microsoft Learn, (2026.), MFC Desktop Applications, https://learn.microsoft.com/en-us/cpp/mfc/mfc-desktop-applications (pristupljeno 29.9.2026.)
2. Jeff Prosise, (1999.) *Programming Windows with MFC second edition,* Microsoft Press
3. Marian Luparu, (2017.), Happy 25th Birthday MFC!, https://devblogs.microsoft.com/cppblog/happy-25th-birthday-mfc/ (pristupljeno 30.9.2026.)
4. Microsoft Foundation Class logo [slika], https://en.wikipedia.org/wiki/File:Microsoft_Foundation_Class_logo.svg (pristupljeno 29.9.2026.)
5. Visual CPP Team, (2007.) ,Quick Tour Of New MFC Functionality, https://devblogs.microsoft.com/cppblog/quick-tour-of-new-mfc-functionality/ (pristupljeno 30.9.2026.)
6. Marius Bancila (2015.), Version history of VC++, MFC and ATL https://mariusbancila.ro/blog/2015/08/12/version-history-of-vc-mfc-and-atl/ (pristupljeno 30.9.2026.)
7. Somasegar's blog, Microsoft Learn, (2024.), https://learn.microsoft.com/en-us/archive/blogs/somasegar/ (pristupljeno 29.9.2026.)
8. WinMerge, (2026), https://winmerge.org/ (pristupljeno 29.9.2026.)
9. Microsoft Learn, (2026.), Windows Presentation Foundation Overview, https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/ (pristupljeno 29.9.2026.)
10. Microsoft Learn, (2026.), WinUI 3, https://learn.microsoft.com/en-us/windows/apps/winui/winui3/ (pristupljeno 29.9.2026.)
11. wxWidgets (2026.) https://wxwidgets.org/about/ (pristupljeno 30.9.2026.)
12. Qt wiki, (2026.), https://wiki.qt.io/List_of_Qt_Applications (pristupljeno 30.9.2026.)
13. Microsoft Learn, (2026.), [slika], https://learn.microsoft.com/en-us/cpp/mfc/hierarchy-chart-categories (pristupljeno 29.9.2026.)

14. Microsoft Learn, (2026.), Document/View Architecture, https://learn.microsoft.com/en-us/cpp/mfc/document-view-architecture (pristupljeno 29.9.2026.)
15. Microsoft Learn, (2026.), C++ Tools and Features in Visual Studio Editions, https://learn.microsoft.com/en-us/cpp/overview/visual-cpp-tools-and-features-in-visual-studio-editions (pristupljeno 30.9.2026.)
16. Izvor slika karata korištenih u programu: Dmitry Fomin, (2013.), Wikimedia Commons, https://commons.wikimedia.org/wiki/File:UNO_cards_deck.svg
17. Microsoft Learn, (2025.), Overview of Graphics, https://learn.microsoft.com/en-us/dotnet/desktop/winforms/advanced/overview-of-graphics

7. SAŽETAK

U ovom radu obrađena je biblioteka Microsoft Foundation Classes (MFC) koja predstavlja objektno-orijentirani okvir za razvoj Windows desktop aplikacija temeljenih na Win32 API-ju. U teorijskom dijelu rada prikazana je povijest razvoja MFC-a od njegovog nastanka početkom 1990-ih godina do današnjeg statusa tehnologije koja se primarno koristi za održavanje postojećih sustava. Također je provedena usporedba MFC-a s novijim Microsoftovim okvirima, poput WPF-a i WinUI-ja, te s nezavisnim rješenjima wxWidgets i Qt.

Nadalje su opisani osnovni koncepti i funkcionalnosti MFC-a, uključujući Windows programski model temeljen na događajima, mehanizam mapiranja poruka, hijerarhiju klasa temeljenu na klasi CObject, dokument/pogled arhitekturu i AFX funkcije. Prikazana je integracija MFC-a s razvojnim okruženjem Visual Studio kroz MFC aplikacijski čarobnjak i Resource Editor.

U praktičnom dijelu rada razvijena je desktop aplikacija koja implementira kartašku igru UNO korištenjem programskog jezika C++, MFC biblioteke i GDI+ grafičkog sustava. Analizirana je struktura aplikacije, korištene MFC komponente i organizacija programskog koda.

Ključne riječi: MFC, Win32 API, Visual Studio, C++, desktop aplikacije, Windows operacijski sustav

8. SUMMARY

This paper covers the Microsoft Foundation Classes (MFC) library, which is an object-oriented framework for developing Windows desktop applications based on the Win32 API. In the theoretical part of the paper, the history of MFC's development is presented, from its inception in the early 1990s to its current status as a technology primarily used for maintaining existing systems. A comparison of MFC with newer Microsoft frameworks, such as WPF and WinUI, and with independent solutions like wxWidgets and Qt was also conducted.

Furthermore, the basic concepts and functionalities of MFC are described, including the event-driven Windows programming model, the Message Maps mechanism, the class hierarchy based on the CObject class, the Document/View architecture, and AFX functions. Features that provide integration of MFC with the Visual Studio IDE through the MFC Application Wizard and Resource Editor are shown.

In the practical part of the paper, a desktop application was developed that implements the card game UNO using the C++ programming language, the MFC library, and the GDI+ graphics system. The application's structure, the MFC components used and the organization of the program code were analyzed.

Keywords: MFC, Win32 API, Visual Studio, C++, desktop applications, Windows operating system